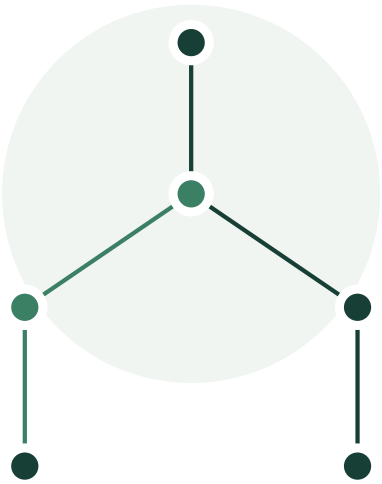
UM GUIA PRÁTICO PARA INICIANTES

Git e GitHub

Versionamento, colaboração e projetos



GIT / GITHUB

Sumário

1 Motivação **1**

2 Conceitos básicos **2**

2.1 O que é versionamento? . . . 2

2.2 Por que versionar? . . . 2

2.3 Git e GitHub são coisas diferentes . . . 3

3 Instalação e configuração **4**

3.1 Instalando o Git . . . 4

3.2 Configurando sua identidade . . . 4

3.3 Criando uma conta e um repositório no GitHub . . . 4

3.4 Configurando uma chave SSH . . . 5

3.5 Conectando a pasta local . . . 5

4 Adicionando mudanças **7**

4.1 Consultando o estado do projeto . . . 7

4.2 Preparando um arquivo com `git add` . . . 7

5 Commitando mudanças **8**

5.1 O que é um commit? . . . 8

5.2 Criando o primeiro commit . . . 8

6 Enviando e buscando mudanças **9**

6.1 Enviando com `git push` . . . 9

6.2 Buscando com `git pull` . . . 9

7 Verificando mudanças **10**

7.1 Limpando o terminal e comparando arquivos . . . 10

7.2 Consultando o histórico . . . 10

8 Resetando mudanças **11**

8.1 Desfazendo preparação ou commit . . . 11

8.2 O comando `checkout` no fluxo antigo . . . 11

9 Gerenciando conflitos **12**

9.1 Quando duas pessoas mudam o mesmo trecho 12
9.2 Concluindo a resolução 12

10 Clonando repositórios públicos **13**
10.1 Explorando projetos 13
10.2 Baixando ou clonando 13

11 Dicas e arquivos importantes **14**
11.1 Autocomplete e atalhos 14
11.2 README: a porta de entrada do projeto 14
11.3 `.gitignore`: arquivos que não entram no repositório 15

12 Dicas finais **16**
12.1 1. Clone projetos interessantes 16
12.2 2. Faça pulls com frequência 16
12.3 3. Não tenha medo de commitar 16
12.4 4. Escreva mensagens claras 16
12.5 5. Envie mudanças com cuidado 16
12.6 6. Contribua com projetos abertos 16
12.7 7. Use o GitHub para acompanhar projetos 16
12.8 8. Mantenha um README informativo 17
12.9 9. Tenha cuidado com vazamentos 17

Capítulo 1

Motivação

O objetivo deste livro é ensinar o básico de versionamento de código sem enrolação e sem termos difíceis, usando Git e GitHub. Ao terminar a leitura, você terá praticado o controle de versões e enviado seu primeiro código para um repositório remoto.

O GitHub também pode funcionar como portfólio público: outras pessoas podem conhecer seus projetos e acompanhar como você trabalha. A plataforma permite que qualquer pessoa contribua com projetos de código aberto, inclusive softwares usados no mundo todo.

Este guia é voltado a iniciantes, estudantes de computação e sistemas de informação e a qualquer pessoa curiosa para aprender Git e GitHub. Os exemplos usam uma pequena pasta de projeto e uma receita de bolo, para que a atenção fique nos comandos e no histórico.

Criar um repositório local, acompanhar mudanças, preparar arquivos, registrar commits, enviar e buscar alterações no GitHub, consultar o histórico e resolver um conflito simples.

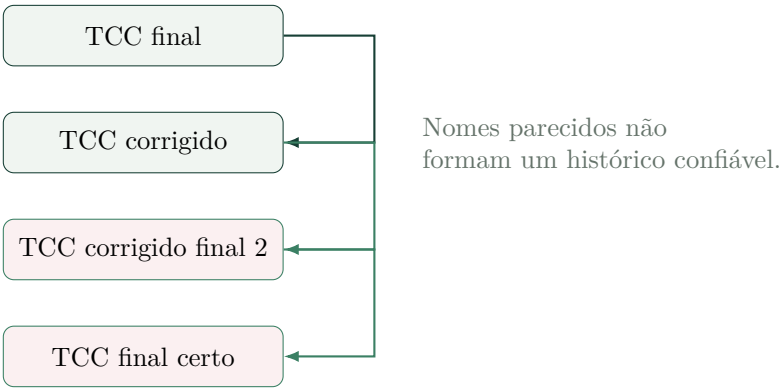
Capítulo 2

Conceitos básicos

2.1 O que é versionamento?

Versionamento é o gerenciamento de versões diferentes de um documento. O documento pode ser um texto, uma planilha ou um código. A ideia é registrar alterações ao longo do tempo para saber o que mudou e, quando necessário, recuperar uma versão anterior.

Pense nas várias cópias de um trabalho de conclusão: *TCC final*, *TCC corrigido*, *TCC corrigido agora vai* e outras parecidas. Sem um histórico claro, fica difícil saber qual arquivo é o mais recente e quais alterações foram feitas em cada um. Em um projeto de software, o problema cresce quando várias pessoas modificam os mesmos arquivos.



2.2 Por que versionar?

Trabalho em equipe
Várias pessoas podem trabalhar no mesmo conjunto de arquivos. O histórico ajuda a localizar alterações e a lidar com conflitos.

Resgate de versões
Se uma mudança introduzir um erro, o histórico permite comparar versões e recuperar um estado anterior para investigar.

Segurança Em projetos hospedados, permissões ajudam a controlar quem pode visualizar ou alterar o código.

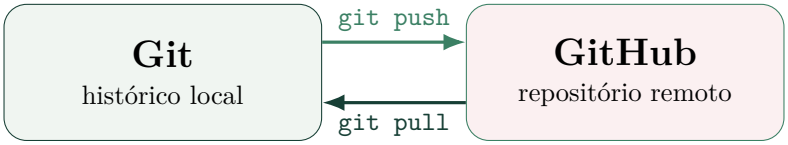
Cópia adicional
O repositório remoto mantém uma cópia do projeto. Controle de versão não substitui uma estratégia completa de backup, mas pode ajudar quando o computador local falha.

2.3 Git e GitHub são coisas diferentes

Git é uma ferramenta de controle de versão. Ele acompanha alterações e mantém um histórico do projeto. É possível usar Git sem criar uma conta no GitHub e sem estar conectado à internet.

GitHub é um serviço online para hospedar repositórios Git e colaborar neles. Ele acrescenta recursos como páginas de projeto, revisão de alterações e controle de acesso.

O GitHub depende do Git, mas o Git pode ser usado sozinho. GitLab e Bitbucket são outros serviços de hospedagem de repositórios. A comparação mais direta é entre esses serviços, e não entre Git e GitHub.



Capítulo 3

Instalação e configuração

3.1 Instalando o Git

Os exemplos deste guia consideram o Windows. Baixe o instalador pela página oficial do Git para Windows e execute a instalação padrão. Durante a instalação, você pode manter as opções recomendadas. No Linux e no macOS, o Git também pode ser instalado pelas ferramentas disponíveis para cada sistema.

Abra a pasta do projeto no Explorador de Arquivos, clique com o botão direito e escolha *Git Bash Here*. No terminal que abrir, confira a instalação:

```
$ git --version
git version 2.x.x
```

3.2 Configurando sua identidade

O Git registra um nome e um e-mail nos commits. Configure os dados que deseja associar às suas alterações:

```
$ git config --global user.name "Seu nome"
$ git config --global user.email "seu@email.com"
```

Confira o nome salvo:

```
$ git config --global user.name
```

O parâmetro `-global` aplica a configuração à sua conta local. O nome e o e-mail podem aparecer nos metadados dos commits que você publicar.

3.3 Criando uma conta e um repositório no GitHub

Crie uma conta no GitHub pelo site e, no menu do perfil, abra *Repositories* e escolha *New*. Um repositório é o espaço que armazena os arquivos e o histórico do projeto. Ele pode conter código, documentos, imagens e outros arquivos.

Escolha um nome curto, como `teste-python`, e decida se o repositório será público ou privado. No exemplo deste livro, ele será público. Deixe as opções de README e `.gitignore` desmarcadas neste primeiro exercício: vamos criar o repositório remoto vazio e conectá-lo à pasta local.

3.4 Configurando uma chave SSH

Uma chave SSH permite que o Git se autentique no GitHub ao enviar ou buscar alterações. O par de chaves contém uma chave privada, que fica protegida no seu computador, e uma chave pública, que será adicionada à sua conta. Antes de criar uma nova, confira se já existem chaves disponíveis:

```
$ ls -al ~/.ssh
```

Se ainda não houver uma chave que você queira usar, crie uma chave Ed25519. Troque o e-mail de exemplo pelo endereço associado à sua conta GitHub:

```
$ ssh-keygen -t ed25519 -C "seu@email.com"
```

Quando o terminal perguntar onde salvar a chave, pressione **Enter** para aceitar o caminho padrão. Se esse caminho já estiver ocupado, não sobrescreva a chave existente: escolha outro nome. Quando solicitado, defina uma frase secreta (*passphrase*) para proteger a chave privada.

Inicie o agente SSH e adicione a chave a ele:

```
$ eval "$(ssh-agent -s)"
$ ssh-add ~/.ssh/id_ed25519
```

Agora, exiba a chave pública:

```
$ cat ~/.ssh/id_ed25519.pub
```

Copie a linha exibida e, no GitHub, abra o menu do perfil, acesse *Settings* e depois *SSH and GPG keys*. Clique em *New SSH key*, dê um título para identificar o computador, selecione o tipo de chave para autenticação e cole a chave pública no campo indicado.

Teste a conexão pelo Git Bash:

```
$ ssh -T git@github.com
```

Se a configuração estiver correta, a resposta incluirá seu nome de usuário e informará que a autenticação funcionou. O GitHub não oferece acesso a um terminal interativo; esse aviso é esperado.

Compartilhe com o GitHub somente o arquivo público `id_ed25519.pub`. Nunca envie nem publique o arquivo privado `id_ed25519`, e não o adicione ao repositório.

As instruções oficiais podem ser consultadas em https://docs.github.com/en/authentication/connecting-to-github-with-ssh.

3.5 Conectando a pasta local

Abra o Git Bash dentro da pasta do projeto. O comando `git init` transforma essa pasta em um repositório local:

```
$ git init
Initialized empty Git repository
```

Na página do repositório do GitHub, copie a URL SSH para conectar um repositório existente. O endereço abaixo é apenas um exemplo:

```
$ git remote add origin git@github.com:curso-git-github/teste-python.git
```

Se você já adicionou o repositório remoto com uma URL HTTPS, pode trocá-la pela URL SSH com:

```
$ git remote set-url origin git@github.com:curso-git-github/teste-python.git
```

O nome `origin` é um apelido convencional para o repositório remoto. A pasta local agora sabe para onde enviar e de onde buscar alterações.

Capítulo 4

Adicionando mudanças

4.1 Consultando o estado do projeto

O comando `git status` mostra em qual branch você está e quais arquivos foram alterados, preparados ou ainda não são acompanhados pelo Git.

```
$ git status
On branch main
No commits yet
nothing to commit
```

Crie na pasta um arquivo chamado `arquivo1.py`, com uma linha simples:

```
print("hello world")
```

Execute `git status` outra vez. O arquivo aparecerá como *untracked*: o Git percebeu que ele existe, mas ainda não o incluiu no próximo registro de mudanças.

4.2 Preparando um arquivo com `git add`

Use `git add` para preparar o arquivo:

```
$ git add arquivo1.py
$ git status
```

O arquivo deixa de aparecer como não acompanhado e passa para a área de preparação, chamada *staging area*. Preparar uma mudança não cria ainda um commit; apenas seleciona o que deverá entrar no próximo commit.

Modificado: o arquivo mudou no computador. **Preparado:** a mudança foi selecionada com `git add`. **Commitado:** a mudança foi registrada no histórico.

Capítulo 5

Commitando mudanças

5.1 O que é um commit?

Um commit agrupa mudanças relacionadas em um registro do histórico. Pense em um pequeno programa que lê um arquivo de texto com uma receita: o projeto precisa tanto do código, por exemplo `codigo-receita.py`, quanto do arquivo `receita-bolo-cenoura.txt`. As duas peças fazem sentido juntas.

Uma mensagem de commit deve resumir a alteração. Uma convenção simples é começar com um verbo, como *Cria leitor da receita* ou *Corrige cálculo do total*.

5.2 Criando o primeiro commit

Prepare os dois arquivos, um comando por linha, e registre o commit:

```
$ git add codigo-receita.py
$ git add receita-bolo-cenoura.txt
$ git commit -m "Cria leitor da receita"
```

O Git precisa saber quem está criando o commit. Se aparecer uma mensagem pedindo identidade, configure nome e e-mail:

```
$ git config --global user.email "seu@email.com"
$ git config --global user.name "Seu nome"
$ git commit -m "Cria leitor da receita"
```

Depois de configurar a identidade, repita o commit. Para confirmar o estado do repositório, use `git status`. Quando tudo estiver registrado, o Git informará que não há nada a commitar.

Faça commits pequenos, cada um com uma mudança coerente. Assim fica mais fácil entender o histórico e localizar a alteração que introduziu um problema.

Capítulo 6

Enviando e buscando mudanças

6.1 Enviando com `git push`

Depois de preparar e commitar, envie os commits para o GitHub:

```
$ git push origin master
```

Uma *branch* é uma linha de desenvolvimento. Em muitos repositórios antigos, a branch principal se chama `master`; em muitos repositórios novos, ela se chama `main`. Use o nome que aparece no seu repositório.

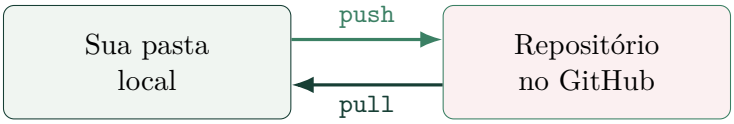
Na primeira publicação, o Git pode pedir autenticação ou indicar que falta configurar a branch remota. Siga as instruções exibidas pelo próprio Git. Ao terminar, abra a página do repositório para conferir os arquivos enviados.

6.2 Buscando com `git pull`

Quando outra pessoa enviar commits para o repositório remoto, atualize sua cópia local com:

```
$ git pull origin master
```

Troque `master` por `main` se esse for o nome da branch principal. Se não houver alterações novas, o Git informará que a cópia já está atualizada. Antes de começar a trabalhar em equipe, faça `git pull` para buscar o estado mais recente.



Capítulo 7

Verificando mudanças

7.1 Limpando o terminal e comparando arquivos

Depois de editar `receita-bolo-cenoura.txt`, por exemplo adicionando uma instrução sobre o tempo de forno, o Git detectará a mudança. No Git Bash, limpe a tela com `clear` e veja as diferenças com:

```
$ clear
$ git diff
```

O comando mostra linhas removidas e adicionadas. Por convenção, linhas adicionadas aparecem com sinal de mais e linhas removidas com sinal de menos. No terminal colorido, elas costumam aparecer em verde e vermelho. Se a mudança já foi preparada com `git add`, use:

```
$ git diff --cached
```

7.2 Consultando o histórico

`git log` lista commits recentes. Cada commit tem um identificador único, chamado *hash*, além de autor, data e mensagem:

```
$ git log
commit 4f8a2c1...
Author: Seu nome <seu@email.com>
Date:   ...

    Cria leitor da receita
```

O hash ajuda a identificar um commit específico. Para ver uma lista resumida por autor, use:

```
$ git shortlog
```

Para ver o conteúdo de um commit, passe seu hash ao `git show`:

```
$ git show 4f8a2c1
```

Use `git status` e `git diff` para conferir o que mudou. Se alguns arquivos já estiverem preparados, confira também `git diff -cached`.

Capítulo 8

Resetando mudanças

8.1 Desfazendo preparação ou commit

Erros acontecem. O Git permite corrigir o que foi preparado ou registrado, mas é importante distinguir essas ações.

Se você preparou um arquivo por engano e quer apenas tirá-lo da área de preparação, use:

```
$ git restore --staged arquivo.txt
```

Em versões antigas do Git, o tutorial de origem usa `git reset <hash>` para mover a referência do histórico a um commit anterior. Uma forma explícita de desfazer o último commit mantendo as mudanças preparadas é:

```
$ git reset --soft HEAD~1
```

Para remover o último commit e deixar as mudanças no diretório de trabalho, mas fora da área de preparação:

```
$ git reset HEAD~1
```

`git restore arquivo.txt` substitui o conteúdo local pela versão preparada. Se houver alterações que você ainda não salvou em outro lugar, elas podem ser perdidas. Confira `git status` e faça uma cópia antes de usar comandos que descartam trabalho.

8.2 O comando `checkout` no fluxo antigo

O tutorial de origem demonstra `git checkout nome-do-arquivo` para recuperar a versão preparada de um arquivo. Nas versões atuais, a forma específica para descartar mudanças locais é `git restore nome-do-arquivo`. Esse comando também descarta as mudanças não preparadas naquele arquivo.

Use `git log` para localizar commits e não compartilhe uma reescrita do histórico com outras pessoas sem combinar com a equipe. Para desfazer uma mudança publicada, em geral é mais seguro criar um novo commit de reversão.

Capítulo 9

Gerenciando conflitos

9.1 Quando duas pessoas mudam o mesmo trecho

Imagine duas pessoas alterando a mesma linha de um arquivo. Uma delas envia seus commits primeiro. Quando a outra tenta fazer `git push`, o Git recusa a publicação porque o repositório remoto já avançou. Primeiro é necessário buscar as mudanças:

```
$ git pull origin master
```

Se as alterações forem em trechos que o Git não consegue combinar, haverá um conflito. O arquivo mostrará as duas versões, delimitadas por marcadores:

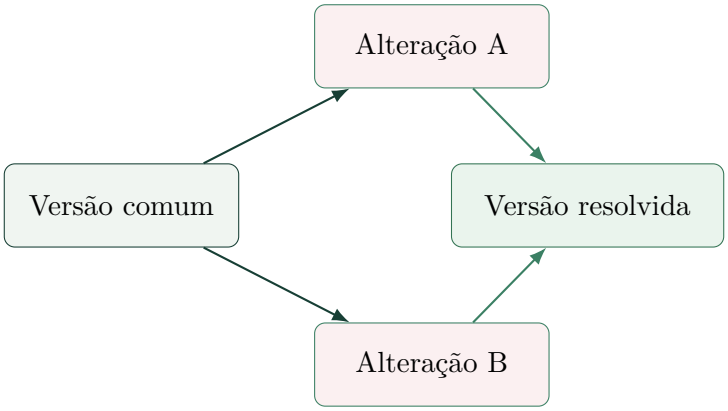
```
<<<<<<< HEAD
open("receita-bolo-cenoura.txt", "r")
=======
open("receita.txt", "r")
>>>>>>> commit-do-outro-colaborador
```

`HEAD` indica a versão que estava na sua branch; a outra parte veio do commit buscado. Leia as duas opções, escolha ou combine o conteúdo correto e apague os marcadores `<<<<<<<`, `=======` e `>>>>>>>`. O arquivo final não deve conter esses marcadores.

9.2 Concluindo a resolução

Depois de salvar o arquivo corrigido, prepare-o, faça um commit e envie:

```
$ git add codigo-receita.py
$ git commit -m "Resolve conflito na leitura da receita"
$ git push origin master
```



Capítulo 10

Clonando repositórios públicos

10.1 Explorando projetos

Repositórios públicos permitem conhecer projetos, estudar como outras pessoas organizam código e, quando possível, contribuir com melhorias. O material apresenta estes exemplos:

- **Build Your Own X:** tutoriais para construir projetos como bots, emuladores e linguagens de programação.
  https://github.com/danistefanovic/build-your-own-x
- **Public APIs:** catálogo de APIs públicas para acessar dados de diferentes serviços.
  https://github.com/public-apis/public-apis
- **The Art of Command Line:** materiais e dicas para trabalhar no terminal.
  https://github.com/jlevy/the-art-of-command-line

10.2 Baixando ou clonando

No GitHub, o botão *Code* permite baixar um arquivo ZIP com o conteúdo do repositório. Essa opção traz os arquivos, mas não cria no computador um histórico Git ligado ao projeto.

Para baixar o projeto com Git, copie a URL de clonagem e execute:

```
$ git clone https://github.com/usuario/repositorio.git
```

O comando cria uma pasta local com os arquivos e o histórico do repositório. Depois, entre na pasta criada para trabalhar:

```
$ cd repositorio
$ git status
```

Capítulo 11

Dicas e arquivos importantes

11.1 Autocomplete e atalhos

No terminal, digite as primeiras letras do nome de um arquivo e pressione **Tab**. O autocomplete pode completar o nome e reduzir erros de digitação.

Para preparar todas as mudanças da pasta atual, o ponto representa o diretório atual:

```
$ git add .
```

Também é possível preparar arquivos e criar o commit em uma sequência curta:

```
$ git add arquivo1.py arquivo2.py
$ git commit -m "Atualiza arquivos do projeto"
```

O atalho `git commit -am` combina preparação e commit para arquivos que já são acompanhados pelo Git:

```
$ git commit -am "Atualiza arquivos acompanhados"
```

`git commit -am` não adiciona arquivos ainda não acompanhados. Para incluir um arquivo novo, execute primeiro `git add nome-do-arquivo`.

11.2 README: a porta de entrada do projeto

O arquivo `README.md` apresenta o projeto para quem abre o repositório. Pense nele como a capa e as instruções iniciais de um livro. Um bom README explica o que o projeto faz, como instalar ou executar, quais tecnologias usa e onde encontrar mais informações.

```
# Nome do projeto

Uma frase explicando o que ele faz.

## Como executar

1. Instale os requisitos.
2. Execute o programa.

## Tecnologias

- Python
- Git
```

Um nome como `teste-python` pode não explicar o objetivo do projeto. Uma descrição clara ajuda visitantes a entenderem o conteúdo.

11.3 `.gitignore`: arquivos que não entram no repositório

O arquivo `.gitignore` lista arquivos e pastas que o Git deve ignorar, como arquivos temporários, ambientes locais e configurações privadas:

```
# Arquivos temporários do Python
__pycache__/
*.pyc

# Ambiente virtual
.venv/

# Configurações locais e segredos
.env
config.local
```

O ponto no início do nome indica que o arquivo costuma ficar oculto em sistemas Unix. No Windows, talvez seja necessário mostrar arquivos ocultos no gerenciador de arquivos.

Um `.gitignore` ajuda a evitar que um arquivo ainda não acompanhado entre no próximo commit. Ele não remove do histórico uma senha já commitada. Se uma senha ou chave foi publicada, revogue-a e gere outra; apagar o arquivo em um commit posterior não torna a credencial antiga segura.

Capítulo 12

Dicas finais

12.1 1. Clone projetos interessantes

Encontrou um repositório interessante? Clone-o, tente entender a estrutura e, se for adequado, execute o código. Você pode aprender com projetos públicos e usá-los como referência para começar algo novo.

12.2 2. Faça pulls com frequência

Se você trabalha com outras pessoas, busque mudanças regularmente. Assim sua cópia local fica mais próxima da versão compartilhada e você percebe conflitos mais cedo.

12.3 3. Não tenha medo de commitar

Commits pequenos e frequentes tornam o histórico mais útil. Evite deixar um projeto inteiro concentrado em apenas um ou dois commits. Registre mudanças coerentes e fáceis de explicar.

12.4 4. Escreva mensagens claras

Evite mensagens vagas como *atualiza código no servidor*. Pergunte a si mesmo: que código, qual parte e para quê? Uma mensagem que continue clara meses ou anos depois ajuda a equipe a entender o histórico.

12.5 5. Envie mudanças com cuidado

Antes de fazer `push`, confira o estado do projeto e execute as verificações disponíveis. Publicar código com erros pode afetar outras pessoas ou sistemas em uso.

12.6 6. Contribua com projetos abertos

Projetos de código aberto permitem que você pratique colaboração e ajude outras pessoas. Contribuições também podem mostrar seu trabalho a quem visita seu portfólio.

12.7 7. Use o GitHub para acompanhar projetos

Siga colegas e projetos que você admira. Acompanhar mudanças em bibliotecas, frameworks e linguagens que você usa é uma forma de aprender como outras equipes trabalham.

12.8 8. Mantenha um README informativo

Não pare no título e em uma frase. Documente como instalar, configurar, executar e testar o projeto. Isso ajuda você, colaboradores e qualquer pessoa que queira entender seu código.

12.9 9. Tenha cuidado com vazamentos

Não commite arquivos com senhas, tokens ou chaves de acesso. Use `.gitignore` para impedir que arquivos locais entrem no repositório e revogue imediatamente qualquer credencial que tenha sido publicada.